Week 1 Formal Verification Report

She Crypts

August 16, 2026

```
theory Xor_Basics
  imports Main
begin
```

1 Boolean Logic and Stream Cipher Primitives

1.1 Exclusive OR Formalization

Exclusive OR (XOR) serves as a core mathematical primitive for symmetric stream ciphers and One-Time Pads. In this module, we formalize XOR logic and verify its algebraic properties.

Definition of XOR We construct XOR from standard conjunction, disjunction, and negation operators: `my_xor a b`.

```
definition my_xor :: "bool ⇒ bool ⇒ bool" where
  "my_xor a b ≡ (a ∧ ¬ b) ∨ (¬ a ∧ b)"
```

Evaluation Checks We test basic truth table outcomes using Isabelle's evaluation engine.

```
value "my_xor True False"
value "my_xor True True"
```

Algebraic Properties First, we establish commutativity for `my_xor`.

```
lemma xor_comm: "my_xor x y = my_xor y x"
  by (auto simp: my_xor_def)
```

Next, we prove the fundamental property for stream cipher decryption: re-applying XOR with key k cancels out the initial encryption step.

```
theorem xor_cancel: "my_xor (my_xor m k) k = m"
  by (auto simp: my_xor_def)
```

The full cancellation theorem is verified in

```
my_xor (my_xor m k) k = m
```

.

end
theory *Modular_Shifts*
  imports *Main*
begin

2 Modular Arithmetic and Shift Ciphers

2.1 Introduction to Shift Encryption

This theory formalizes a classical modular shift cipher operating over natural numbers. We define the encryption transformation function and verify that sequential shifts compose additively.

Encryption Function Definition We define encryption $E(n, k, m) = (m + k) \pmod{n}$ where n is the alphabet size, k is the shift key, and m is the message token.

definition *E :: "nat ⇒ nat ⇒ nat ⇒ nat"* **where**
  "E n k m ≡ (m + k) mod n"

Concrete Test Evaluation We evaluate the encryption function on sample concrete inputs to verify functional behavior.

value *"E 26 3 5"*

Composition Verification We prove that double encryption under key k_1 followed by key k_2 is equivalent to a single encryption under key $k_1 + k_2$.

theorem *shift_compose: "E n k2 (E n k1 m) mod n = E n (k1 + k2) m mod n"*
  by *(simp add: E_def mod_simps algebra_simps)*

The formal property verified by the Isabelle kernel is

E n k2.0 (E n k1.0 m) mod n = E n (k1.0 + k2.0) m mod n

.

end